

4. Common CNN Architectures

Task 4.1: Tiny-ImageNet with VGG16

- Fine-tuned VGG16 on Tiny-ImageNet. Best test accuracy reported in Table 1. Training/validation accuracy curves show convergence.
- Figure 1 reports the training and validation accuracy curves for the VGG16 run in Task 4.1.
- The final score indicates reasonable transfer from ImageNet pretraining, while still leaving room for gains from longer training and stronger augmentation.

Table 1: Tiny-ImageNet: best test accuracy.

Metric	Value
Test Accuracy (%)	38.27

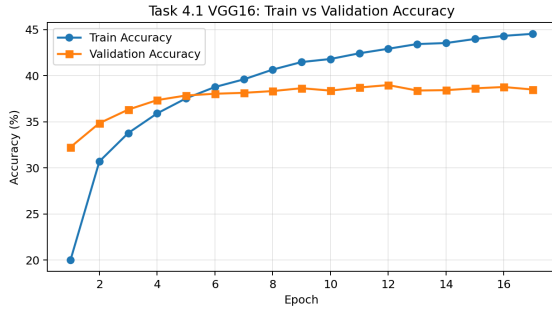


Figure 1: Task 4.1 VGG16 training vs. validation accuracy on Tiny-ImageNet.

Task 4.2: ConvNeXt Model Scaling

- Compared ConvNeXt variants (tiny, small, base) on Tiny-ImageNet. Accuracy and MSE vs. number of parameters (Figure 2).
- Scaling increases capacity and generally improves accuracy, but with diminishing returns relative to parameter growth.

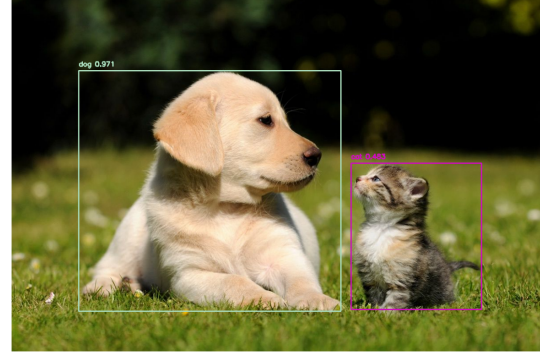


Figure 2: ConvNeXt scaling: accuracy and MSE vs. number of parameters.

- Larger windows overfit temporal context in this setup, increasing test error despite lower training RMSE.

Table 2: RMSE vs. window size for LSTM regression.

Window Size	Train RMSE	Test RMSE
1	23.22	50.82
3	22.28	56.25
6	21.90	57.57
12	17.68	81.94

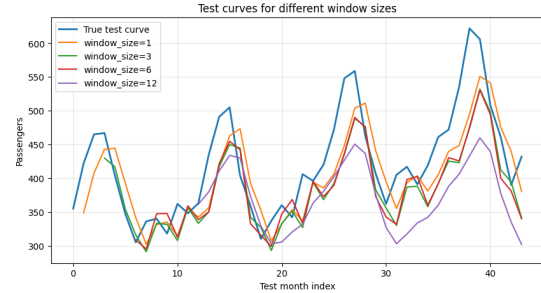


Figure 3: Test curves for different window sizes (Task 5.1).

5. RNN

Task 5.1: RNN Regression

- Window size sweep: RMSE vs. window size (Table 2). Best window size = 1 for lowest test RMSE. Test curve plot in Figure 3.

Task 5.2: Text Embeddings

- Compared Embeddings, LSTM, and LSTM+GloVe models. Test accuracy and sentiment scores (Table 3). Accuracy/loss curves in Appendix.
- Pretrained embeddings improve sentiment polarity separation and recover the best overall accuracy.

Table 3: Text embedding models: test accuracy and sentiment scores.

Model	Test Acc (%)	Neg Score	Pos Score
Embeddings	78.00	0.48	0.48
LSTM	73.28	0.25	0.25
LSTM+GloVe	78.21	0.34	0.66

Task 5.3: Text Generation

- BLEU score vs. temperature for char-level and word-level models (Table 4, Figure 4).
- Moderate sampling temperature gives the best BLEU balance; very high temperature hurts coherence.

Table 4: BLEU score vs. temperature.

Temperature	Char BLEU	Word BLEU
0.0	0.0144	0.0116
0.5	0.0177	0.0111
1.0	0.0133	0.0115
2.0	0.0115	0.0078

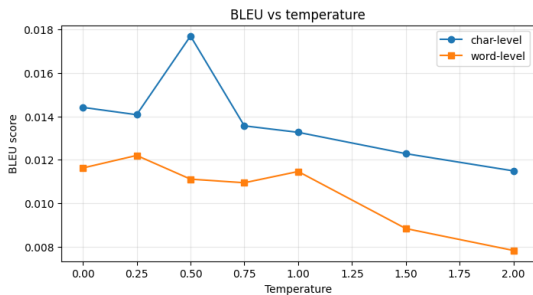


Figure 4: BLEU vs. temperature for char-level and word-level generation.

6. Autoencoders

Task 6.1: Representation Learning

- Autoencoder for representation learning. Classifier accuracy and MSE for linear AE, conv AE, and PCA (Table 5).
- Learned nonlinear representations slightly outperform PCA in classification, while reconstruction quality remains comparable.

Task 6.2: Custom Loss Functions

- Effect of custom loss terms on denoising MSE (Table 6, Figure 5).

Table 5: Representation learning: classifier accuracy and MSE.

Model	Val Acc (%)	MSE
Linear AE	81.30	0.026
Conv AE	81.69	0.029
PCA (10 comp)	81.38	0.0256

- The table compares denoising quality under different objectives using a unified MSE evaluation protocol.

Table 6: Denoising MSE for different loss functions.

Loss	Test MSE
MSE	—
MAE	—
SSIM	—

Noisy Input VS Reconstructed VS Clean Image

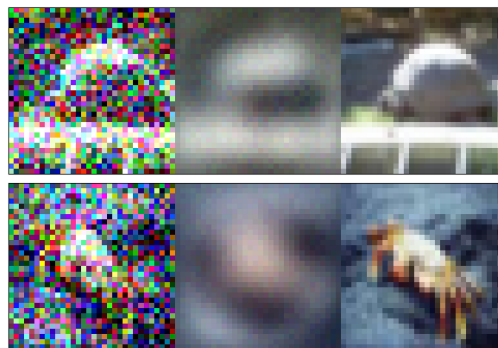


Figure 5: Denoised image examples for different loss functions.

7. VAE GAN

Task 7.1: MNIST Generation (VAE/-GAN)

- VAE: reconstruction MSE and Inception Score. GAN: Inception Score. Table 7.
- Removing KL improves reconstruction MSE but harms generative quality (IS), showing the expected trade-off between reconstruction fidelity and latent regularization.

Task 7.2: MAE vs. cGAN Colourisation

- Mean MAE for MAE and cGAN on CIFAR-10 colourisation (Table 8). Qualitative examples in Appendix.

Table 7: VAE vs. GAN: MSE and Inception Score on MNIST.

Model	MSE	Inception Score
VAE (latent=10 + KL)	10.983575	5.342967
VAE (no KL)	8.836834	2.374818
GAN (latent=10)	N/A	1.000000

- Pixel-wise MAE is lower for the MAE model, while cGAN may still produce perceptually sharper colours in selected examples.

Table 8: MAE vs. cGAN: mean MAE on CIFAR-10 colourisation.

Model	Mean MAE
MAE	0.0899
cGAN	0.0970

8. RL

Task 8.1: Q-learning

- Q-learning (and SARSA) with ϵ -greedy and Softmax policies. Average reward (last 50 episodes) vs. training episodes (Figure 6).
- ϵ -greedy converges faster in this experiment, while Softmax can be smoother but more sensitive to temperature tuning.
- Appendix includes (i) key modifications to DQNAgent for the four agents and (ii) secondary behavioural plots.

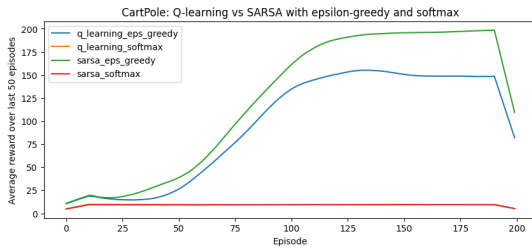


Figure 6: Q-learning: average reward (last 50 episodes) vs. training episodes.

9. Diffusion Model

Task 9.1: Forward Noising

- Diffusion qualitative result: real and generated MNIST samples (Figure 7). Training loss curve and training code are included in Appendix Task 9.2.

- Visual inspection confirms that increasing diffusion steps progressively destroys semantic structure while preserving the stochastic forward-process formulation.



Figure 7: Diffusion model qualitative output: real vs. generated MNIST digits.

10. Data Classification YOLO

Task 10.1: Roof Classification

- Finetuned yolov8n-cls.pt on aerial roof images (flat vs. pitched). Dataset: course-provided aerial data merged with Zenodo BRAILS roof-type images.
- Data augmentation: RandAugment, CutMix, MixUp, Random Erasing.
- Training: epochs=30, batch=64, imgsz=320, patience=10. Best checkpoint at epoch 26.
- Test accuracy and learning curves (Figure 8, Table 9).
- Top-5 accuracy reaches 100%, indicating class separability is high even when top-1 mistakes remain.

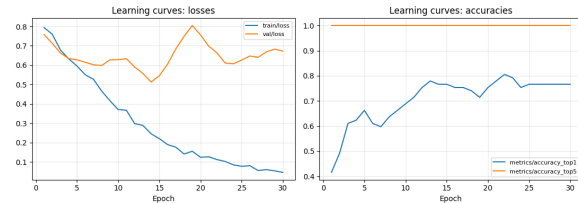


Figure 8: YOLOv8 roof classification: training loss and accuracy curves.

Table 9: YOLOv8 roof classification: test metrics.

Metric	Value
Test Top-1 Accuracy (%)	83.61
Test Top-5 Accuracy (%)	100.0
Best checkpoint	epoch 26

Appendix

Task 5.2: Accuracy and loss curves

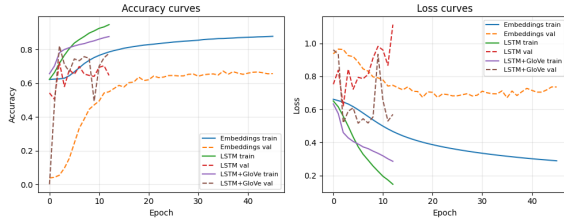


Figure 9: Task 5.2 training and validation curves for Embeddings, LSTM, and LSTM+GloVe models.

Task 7.2: MAE vs. cGAN qualitative comparison

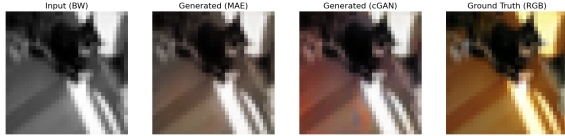


Figure 10: Task 7.2 qualitative colourisation examples for MAE and cGAN models.

Task 8.1: DQNAgent modifications and secondary plots

Key modifications to DQNAgent:

- Added an algorithm switch (q_learning/sarsa) to control TD target computation.
- Added a policy switch (epsilon_greedy/softmax) with temperature-controlled softmax action selection.
- Extended replay memory tuples to include next_action, enabling on-policy SARSA updates.
- Logged behavioural diagnostics during training: greedy-action fraction, policy entropy, and average TD-error magnitude.

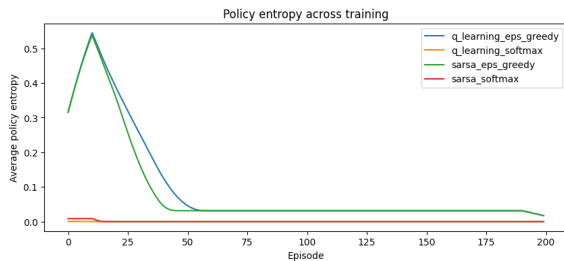


Figure 11: Task 8.1 secondary plot: policy entropy across training for the four agents.

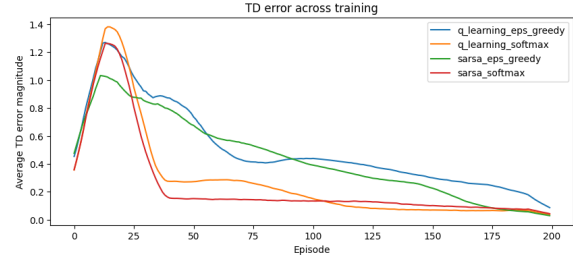


Figure 12: Task 8.1 secondary plot: TD error magnitude across training for the four agents.

Task 9.2: Diffusion training code and loss curve

```
# Training loop for the diffusion model
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for x, _ in train_loader:
        x = x.to(DEVICE)
        t = torch.randint(0, T, (x.size(0),), device=DEVICE).long()
        x_noisy, noise = add_noise(x, t, sqrt_alphas_cumprod,
                                   sqrt_one_minus_alphas_cumprod)
        pred_noise = model(x_noisy, t)
        loss = F.mse_loss(pred_noise, noise)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * x.size(0)
```

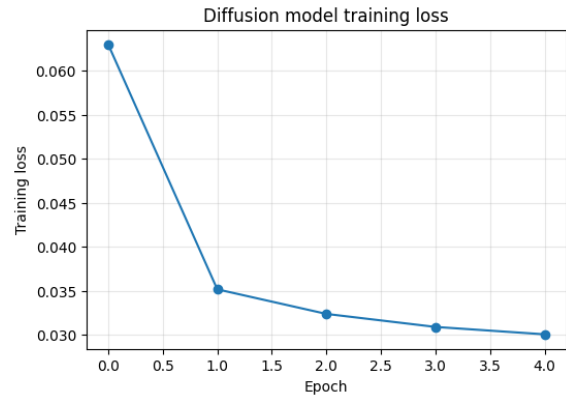


Figure 13: Task 9.2 diffusion training loss across epochs.